핵심 테이블 설계서 (10개)

AI 채용 플랫폼 · 작성 2026-11-10 · 전체 테이블 50개 중 중심이 되는 10개만 설명합니다. 근거는 마이그레이션 (`backend/migrations/versions/*.py`), `backend/app/models.py`, `docs/trd/03-data-model.md` 이며, 실행 중인 DB에 접속해 확인한 것은 아닙니다.

1. 한눈에 보기

이 시스템은 **공고 → 지원 → 면접 → 평가 → 결과** 한 줄로 흐르고, 그 흐름의 가운데에 **지원 건(`applications`)**이 있습니다. 나머지 40개 테이블은 이 10개 중 하나에 매달린 보조 기록입니다(3절).

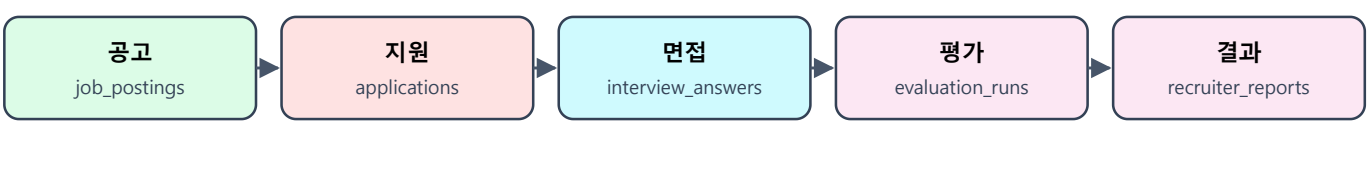


그림 1. 데이터가 흐르는 순서와 각 단계의 대표 테이블.

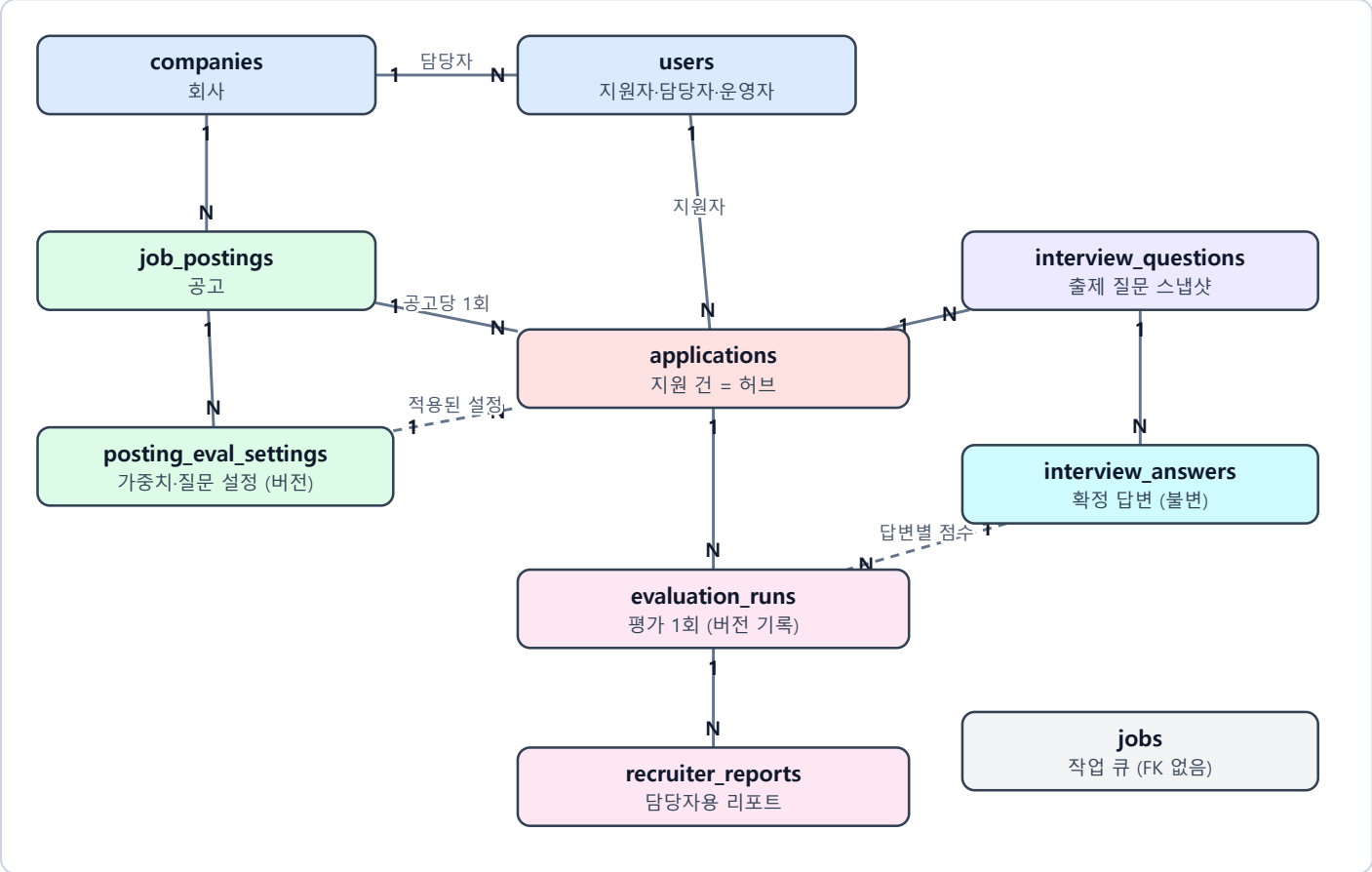


그림 2. 핵심 10개의 관계. 선은 "1개(1)가 여러 개(N)를 가진다"는 뜻이고, 점선은 비어 있을 수 있는 연결입니다. `jobs`만 지원 건과 외래키로 이어져 있지 않습니다. 선은 마이그레이션 SQL의 `REFERENCES`에서 확인했습니다(그림이 복잡해지지 않게 일부 선은 생략).

2. 10개 테이블

어떻게(칼럼·연결)와 **왜**(이유)를 함께 적었습니다. "확인"은 코드·마이그레이션에서 직접 본 것이고, "해석"은 이유가 문서에 없어 제가 추론한 것입니다.

1. `companies` — 회사

핵심 칼럼	`name`, `intro`, `biz_no`, `verification` (승인 대기·승인·반려), `suspended_at`
연결	`users`, `job_postings`, `company_knowledge`, `notifications` 등이 가리킴
왜 이렇게	공고와 담당자를 회사 단위로 묶는 뿌리입니다. 승인 전 회사는 공고를 게시하지 못하게 `verification` 하나로 막습니다.
제약·주의	정지(`suspended_at`)하면 공고가 내려갑니다. 회사 격리의 기준이 되는 값이라 거의 모든 회사 소유 테이블이 `company_id` 를 복사해 갖습니다.

확인: `models.py`, 마이그레이션 FK

2. `users` — 사용자 (한 테이블, 역할로 구분)

핵심 칼럼	`role` (지원자·담당자·운영자), `company_id` (담당자만), `email`, `password_hash` (Argon2id), `status`, `withdrawn_at`
연결	`companies` 를 가리키고, 세션·동의·이력서·지원 건이 이 테이블을 가리킴
왜 이렇게	세 종류의 사람을 테이블 3개로 나누지 않고 `role` 하나로 구분합니다. 로그인·세션·권한 검사 코드를 한 벌만 쓰게 하려는 것으로 보입니다(해석).
제약·주의	담당자는 `company_id` 가 반드시 있어야 하는 CHECK가 있습니다(문서 03). **불일치**: 문서 03은 생년월일·성별 칼럼이 없다고 하지만 `models.py` 에는 `birth_date`, `gender` 가 있습니다("AI 입력에는 복사하지 않음" 주석).

확인: `models.py` · 문서 03

3. `job_postings` — 공고

핵심 칼럼	`status` (작성 중·게시·마감), `title`, `job_family`, `duties`·`requirements`·`preferred`, `deadline_at`, `coding_*` 6개, `current_settings_id`, `settings_locked_at`, `hidden_at`, `deleted_at`, `row_version`
연결	`companies` 아래에 있고, 지원 건·설정·회사 자료·동의 기록이 이 테이블을 가리킴
왜 이렇게	게시 전에는 비어 있어도 되지만 게시 이후에는 제목·직무·마감이 필수인 CHECK로 "초안과 게시본"을 한 테이블에서 다룹니다. 삭제·숨김은 행을 지우지 않고 시각 칼럼(`deleted_at`, `hidden_at`)으로 표시합니다.
제약·주의	첫 면접이 시작되면 `settings_locked_at` 이 찍히고, 트리거(`t_posting_coding_lock`)가 코딩 설정과 현재 설정 변경을 막습니다. `row_version` 으로 동시 수정을 감지합니다.

확인: `models.py` · 문서 03 §5

4. `posting_eval_settings` — 평가·질문 설정 (버전별)

핵심 칼럼	`version`, `w_expertise`·`w_logic`·`w_attitude`·`w_delivery`·`w_coding`, `custom_items`, `question_plan`, `think_sec`, `answer_sec`, `followup_max`, `rubric_version`
연결	`job_postings` 의 자식. 지원 건·평가 run이 "어느 버전을 썼는지" 가리킴
왜 이렇게	설정을 수정하지 않고 **새 버전 행을 추가**합니다. 이미 응시한 사람의 점수가 어떤 가중치로 나왔는지 나중에도 설명할 수 있어야 하기 때문입니다(`CLAUDE.md` §6 "가중치 버전 기록").
제약·주의	수정·삭제 금지 트리거(`t_settings_immutable`)와 "가중치 합 = 100" CHECK가 있습니다(문서 03). 공고가 잠긴 뒤에는 새 행 추가도 막힙니다(`t_settings_locked`).

확인: 마이그레이션 트리거 · `models.py`

5. `applications` — 지원 건 — 시스템의 허브

핵심 칼럼	`attempt_status` · `eval_status` · `decision_status` (상태 3개), `resume_snapshot`, `resume_snapshot_text`, `consent_record_id`, `eval_settings_id`, 시각 칼럼 10개, `share_result`, `*_purged_at`
연결	지원자(`users`)·공고·동의 기록·설정 버전·이력서 PDF를 가리키고, 질문·면접·코딩·평가·결과 테이블 대부분이 이 테이블을 가리킴
왜 이렇게	① **허브**: 한 사람이 한 공고에 지원한 사건 하나를 기준으로 모든 기록을 묶어, 보관 삭제·권한 확인·결과 공개를 지원 건 단위로 합니다. ② **상태 3분리**: 응시·평가·결정을 따로 둬서 평가가 실패해도 응시 기록이 남게 합니다(`CLAUDE.md` §6). ③ **이력서 사본**: 지원 시점 이력서를 JSON으로 복사해, 나중에 지원자가 고쳐도 질문 근거가 변하지 않게 합니다.
제약·주의	`t_guard_application` 트리거가 허용되지 않은 상태 이동을 DB에서 막습니다. 한 공고에 한 번만 지원(`UNIQUE(posting_id, candidate_id)`, 문서 03 기준).

확인: `models.py` · 마이그레이션 트리거 · 문서 03

6. `interview_questions` — 출제된 질문 (스냅샷)

핵심 칼럼	`seq`, `role` (본질문 MAIN·꼬리 FOLLOWUP), `parent_id`, `slot_type`, `source`, `text`, `grounding` (JSON), `answer_limit_sec`, `prompt_version`
연결	`question_sets` · `applications` 의 자식. 꼬리질문은 `parent_id` 로 자기 테이블의 본질문을 가리킴
왜 이렇게	질문 은행이 아니라 **이 지원자에게 실제로 한 질문의 사본**입니다. 은행이나 공고가 나중에 바뀌어도 "그때 무슨 질문을 했고 어떤 근거로 만들었는지(`grounding`)"가 남습니다. 본질문·꼬리질문을 한 테이블에 두고 `role` 로 구분합니다.
제약·주의	수정·삭제 금지 트리거(`t_iq_immutable`). 보관 삭제만 예외입니다. `source` 의 허용 값이 마이그레이션 0034(이번 변경, **아직 DB에 적용하지 않음**)에서 `TEMPLATE` 이 추가됩니다.

확인: `models.py` · 마이그레이션

7. `interview_answers` — 확정된 답변 (불변)

핵심 칼럼	`transcript` (지원자가 본 자막), `eval_text` (평가 기준 텍스트), `audio_key`, `end_reason` (버튼·시간 만료), `is_no_answer`, `stt_failed`, `code_text`
연결	`interview_turns` (제시 1회)· `interview_questions` · `applications` 를 가리키고, 평가 점수 행이 이 테이블을 가리킴
왜 이렇게	턴(진행 중, 바뀜)과 답변(확정, 안 바뀜)을 나눴습니다. **확정된 것만** 답변 테이블에 넣어, "평가의 근거가 되는 원문"이 이후에 바뀌지 않게 합니다. 자막(`transcript`)과 평가용 텍스트(`eval_text`)를 따로 두는 이유는 문서 03에 "비식별화본, 평가·인용 기준"으로 적혀 있습니다.
제약·주의	수정·삭제 금지 트리거(`t_answers_immutable`). 문항당 확정 답변 1개(`UNIQUE(question_id)`, 문서 03 기준).

확인: `models.py` · 트리거 · 문서 03

8. `evaluation_runs` — 평가 1회

핵심 칼럼	`run_no`, `status`, `model_name`, `model_digest`, `prompt_version`, `rubric_version`, `scoring_version`, `eval_settings_id`, `job_id`, `lease_token`
연결	`applications` 의 자식. 자식으로 `answer_evaluations` (답변×항목 점수·근거)와 리포트 2종
왜 이렇게	다시 평가하면 기존 행을 고치지 않고 **run을 하나 더 만듭니다**. 어떤 모델·프롬프트·루브릭·가중치로 계산했는지가 run마다 남아 점수를 재현·비교할 수 있습니다(`CLAUDE.md` §6).
제약·주의	점수 행(`answer_evaluations`)은 수정 금지 트리거가 걸려 있습니다. 평가 실패는 run의 `status` 로 남고 응시 기록을 지우지 않습니다.

확인: `models.py` · 트리거

9. `recruiter_reports` — 담당자용 리포트 (지원자용은 `candidate_feedback`)

핵심 칼럼	`total_score`, `score_*` (항목별), `question_scores` (JSON), `weights` (적용 가중치 사본), `summary`, `strengths`, `improvements`, `insufficient_count`
연결	`evaluation_runs` · `applications` 의 자식(지원 건당 1개)
왜 이렇게	점수는 **서버 코드가 계산해서 저장**한 결과입니다(LLM이 총점을 정하지 않음). 가중치 사본을 같이 저장해 나중에 설정이 바뀌어도 그때 점수가 설명됩니다. 지원자에게 보이는 `candidate_feedback` 은 별도 테이블이고, 총점 칼럼이 없습니다(`models.py` 비교).
제약·주의	공개 범위를 테이블로 분리해, 지원자 API가 내부 정보를 실수로 내보낼 가능성을 줄이려는 구조입니다(해석).

확인: `models.py` · 해석: 분리 이유

10. `jobs` — 영속 작업 큐

핵심 칼럼	`kind`, `payload` (ID만), `application_id`, `dedupe_key`, `resource` (LLM·RUNNER·CPU·NONE), `state`, `attempts`, `lease_owner` · `lease_token` · `lease_expires_at`, `depends_on`
연결	**자기 자신(`depends_on`)만** 외래키로 가리킴. `application_id` 에는 FK가 없음
왜 이렇게	평가·질문 생성·음성 합성 같은 오래 걸리는 일을 DB 행으로 둡니다. 호출한 쪽의 트랜잭션에서 등록해 "제출 저장과 작업 등록을 원자적으로" 하고(`CLAUDE.md` §6), 재시작해도 남아 있게 합니다. `dedupe_key` 로 중복 등록을, 임대(60초)와 `lease_token` 으로 중복 실행을 막습니다.
제약·주의	FK가 없는 이유는 문서에 적혀 있지 않습니다. 큐 행이 지원 건 삭제와 엮이지 않게 한 것으로 보입니다(해석). 문서 03의 ERD는 `applications → jobs` 선을 그리지만 SQL에는 그 FK가 없어서 **불일치**입니다.

확인: `models.py` · 마이그레이션 · 해석: FK 없는 이유

3. 나머지 40개는 어디에 매달려 있나

영역	개수	테이블 (핵심 10개 제외)
계정	4	`sessions`, `password_reset_tokens`, `signup_codes`, `auth_throttle`
공고	3	`posting_templates`, `coding_problems`, `coding_testcases`
동의·이력서	4	`consent_documents`, `consent_records`, `resume_files`, `resumes`
지원	2	`state_transitions`, `application_state_events`
질문	5	`question_bank`, `question_examples`, `company_knowledge`, `question_sets`, `question_tts_assets`
면접	5	`interviews`, `interview_slots`, `interview_turns`, `followup_decisions`, `proctor_events`
코딩	4	`coding_attempts`, `coding_runs`, `coding_submissions`, `coding_test_results`
평가	5	`answer_evaluations`, `answer_pre_evaluations`, `candidate_feedback`, `decision_history`, `recruiter_notes`
소통	5	`interview_schedules`, `recruiter_mails`, `notifications`, `inquiry_threads`, `inquiry_messages`
운영	3	`audit_logs`, `periodic_tasks`, `file_deletions`

예: 면접 진행(`interviews`, `interview_turns`, `followup_decisions`)은 `interview_questions`·`interview_answers` 곁에서 진행 상태를 기록하고, 코딩 응시 4개(`coding_*`)는 `applications`에 매달려 있으며, `candidate_feedback`은 `recruiter_reports`와 짝입니다.

4. 구조를 더 단순화할 수 있나

가능하지만 이득이 작습니다(제 판단). 테이블을 합치려면 마이그레이션·트리거·CHECK·복합 FK·테스트를 같이 고쳐야 합니다.

- **가장 현실적**: `question_bank`와 `question_examples` 병합(둘 다 "질문 + 임베딩"). 이번에 질문을 은행에서 뽑지 않게 바꿔 은행의 역할이 줄었습니다.
- **유지 권장**: 불변 테이블(`interview_answers` 등)과 진행 테이블(`interview_turns`)의 분리, 담당자용·지원자용 리포트 분리, `state_transitions`(트리거가 읽는 참조표).
- **더 안전한 정리**: 테이블 수보다 넓은 테이블의 칼럼입니다. `job_postings`의 `coding_*` 6개, `applications`의 시각 칼럼 10개. 다만 CHECK·트리거와 엮여 있어 같이 고쳐야 합니다.

5. 관계선이 없는 테이블은 왜 그런가

테이블	이유	근거
`auth_throttle`	키가 이메일 해시와 IP입니다. 가입 안 한 이메일의 실패도 세야 해서 `users`를 가리킬 수 없습니다.	확인 + 해석
`signup_codes`	가입 **전**에 코드를 보내므로 아직 사용자 행이 없습니다.	확인
`audit_logs`	대상이 `target_type` + `target_id`(텍스트)로 여러 종류를 가리키는 구조이고, 삭제 불가라 FK를 걸면 데이터 삭제에 걸림돌이 됩니다.	확인(구조) + 해석(이유)
`jobs`	자기 자신만 참조합니다(앞의 10번 카드).	확인 + 해석
`state_transitions`	허용되는 상태 이동 값을 담은 참조표입니다. 트리거가 읽습니다.	확인
`periodic_tasks`, `file_deletions`	운영용 일정표와 파일 삭제 대기열입니다. 지울 파일이 가리키던 행은 이미 사라졌으므로 FK가 있으면 안 됩니다.	문서 + 해석
`question_examples`	회사·지원 건과 무관한 전역 참고 질문입니다.	확인

참고: `models.py`에는 외래키 선언이 거의 없고 실제 관계는 마이그레이션 SQL에 있어서, 모델만 보면 관계가 없는 테이블이 더 많아 보입니다.

6. 확인하지 못한 것

- 실행 중인 DB에 접속해 보지 않았습니다. 마이그레이션 0001~0034와 문서·모델을 읽은 결과이며, 이후 마이그레이션에서 바뀐 CHECK 세부는 현재와 다를 수 있습니다.
- FK 연결 대상은 SQL을 정규식으로 읽어 확인했습니다. 칼럼명과 개수는 정확하지 않을 수 있습니다.
- `UNIQUE` 제약과 "합 = 100" CHECK는 문서 03을 근거로 썼고 SQL 줄 단위로 대조하지 않았습니다.
- `CLAUDE.md` §0은 멀티테넌시를 제외한다고 하지만 스키마에는 회사 격리용 `company_id`와 복합 FK가 있습니다. 코드가 구현된 대로 적었습니다.